Machine Coding: Accordion

- **Problem:** Build a reusable `<Accordion items allowMultiple defaultOpenIds>`. Each item has a header and a panel. Clicking a header expands or collapses its panel with a smooth height animation. By default only one panel is open at a time (opening one closes the other); with `allowMultiple` any number can be open. It must work with the keyboard (Enter / Space to toggle, Arrow Up / Down / Home / End to move between headers), be announced correctly by screen readers, and handle panels of any height.
- **Tech stack:** React 19 + Vite (JavaScript, plain CSS)
- **Time limit:** 60 minutes

PART A: Thought process

General steps for any machine coding task

1. **Clarify** the requirements. Don't guess; ask.
2. **List the interactions.** Every user action and what it should do becomes your checklist.
3. **Sketch the architecture.** Components, hooks and modules, with one job each.
4. **Design the state.** Find the minimum you have to store; derive the rest.
5. **Define the data and contracts.** Example data and the API shape (mock it if there's no API).
6. **Build the simplest working version first,** then add one working layer at a time.
7. **Handle every state and edge case:** empty, loading, error, success.
8. **Wrap up.** Explain the trade-offs and what you'd add with more time.

Time budget (60 min)

Phase	Time	Output
Requirements + clarifying questions	5 min	Single vs multiple, animation, a11y rules
Use cases + props contract + state	5 min	`<Accordion>` API, `openIds: Set`
Static list + click to show/hide (no animation)	8 min	Working accordion
Single / multiple mode with a pure `toggleId`	7 min	Both modes work
Semantics: `<button>`, `aria-expanded`, `aria-controls`, region, `useId`	8 min	Screen-reader friendly
Height animation (grid `0fr -> 1fr`) + `inert`	10 min	Smooth, any content height
Keyboard navigation between headers	7 min	Arrow / Home / End
Empty state, reduced motion, polish	5 min	Edge cases covered
Wrap-up	5 min	Trade-offs explained

Tip: think out loud

Say what you're doing and why, before you type it: "*I'll keep the open ids in a Set so single and multiple mode use the same state shape.*" The interviewer grades your reasoning as much as your code. If you get stuck, saying where you're stuck lets them help.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Render a list of items, each with a **header** (always visible) and a **panel** (content).
- Clicking a header toggles its panel open / closed.
- **Single mode (default):** opening one panel closes the one that was open. Clicking the open header closes it (all can be closed).
- **Multiple mode (`allowMultiple`):** each panel toggles independently.
- `defaultOpenIds` decides which panels start open.
- Expand / collapse with a **smooth height animation**, for content of any height.
- A visual indicator (chevron) shows open vs closed.

Non-functional requirements

Area	Requirement
Accessibility	Headers are real `<button>`s inside headings; `aria-expanded`, `aria-controls`, panel `role="region"` + `aria-labelledby`; collapsed content can't be tabbed into; WAI-ARIA keyboard pattern
Performance	Toggling one item re-renders cheaply; animation runs on the compositor-friendly CSS path, no JS measuring per frame
Robustness	Works for content that's short, long, or changes height after opening (images, wrapped text on resize)
Reusability	One component, driven by data (`items`) and props; works multiple times on one page (unique ids)
Usability	Respects `prefers-reduced-motion`; visible focus ring; large click target (whole header row)
Extensibility	Easy to add controlled mode, disabled items, icons, nested accordions

Clarifying questions to ask

1. Can **more than one** panel be open at once, or only one? Should it be configurable?
2. Can the user **close all** panels, or must one always stay open?
3. Is the content **static text** or can it be any JSX (links, images, forms, even another accordion)?
4. Do we need **keyboard support** beyond Tab + Enter (arrow keys between headers)?
5. Should open state be **controlled by the parent** (e.g. saved in the URL / local storage) or is internal state fine?
6. Is an **animation** required? Any library allowed (framer-motion is installed), or plain CSS?

Assumptions we will build with

- Single mode by default, `allowMultiple` prop turns on multiple mode. All panels **can** be closed.
- Content is any React node; the demo uses text.
- Full WAI-ARIA accordion keyboard pattern (Enter / Space, Arrow Up / Down, Home / End).
- Internal (uncontrolled) state with `defaultOpenIds`; controlled mode is a follow-up.

- Plain CSS animation, no library.

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads	Items render; ids in `defaultOpenIds` are open (only the first one in single mode)
Click a closed header (single mode)	It opens with an animation; the previously open panel closes
Click a closed header (multiple mode)	It opens; other panels stay as they are
Click an open header	It closes
Tab	Focus moves header to header; skips content of **collapsed** panels; enters content of **open** panels
Enter / Space on a focused header	Toggles it (free with a native `<button>`)
Arrow Down / Arrow Up on a header	Focus moves to the next / previous header, wrapping at the ends
Home / End on a header	Focus moves to the first / last header
Screen reader on a header	Reads "What is React?, button, expanded" (or "collapsed")
Toggle "Allow multiple open" off while 3 are open	Accordion resets to its default state (only one open)
`items` is empty	Shows "Nothing to show yet." instead of an empty box
Panel content is very long / grows after opening	Panel shows all of it, never clipped
User has reduced motion enabled	Panels open / close instantly
Click rapidly while animating	Animation reverses smoothly from where it is, no jump

This table is the checklist for the build and for testing at the end.

3. Design the architecture

Module	Responsibility
`App`	Demo page: holds the "Allow multiple open" checkbox, passes data to `Accordion`
`Accordion`	Owns which items are open, handles keyboard navigation between headers, renders the list
`AccordionItem`	Pure presentation of one item: heading button + panel, wires the ARIA ids
`utils/accordion.js`	Pure functions: `toggleId` (open/close rule) and `nextIndex` (keyboard rule)
`data/faq.js`	Example data with stable ids
`App.css`	Layout, chevron, and the height animation

```
App
 └─ Accordion  (state: openIds, headerRefs)
```

```
     ├─ AccordionItem  (isOpen, onToggle, onKeyDown, headerRef)
     │   ├─ h3 > button   (aria-expanded, aria-controls)
     │   └─ div[role=region] (aria-labelledby, inert when closed)
     ├─ AccordionItem
     └─ ...
utils/accordion.js  ← used by Accordion (no React, easy to test)
```

Start with everything in one component. Split out `AccordionItem` as soon as each item needs its own `useId` (hooks can't be called inside `.map()`), and move the rules into `utils` once they're worth testing on their own.

4. Design the state / data model

State or field	Purpose
`openIds: Set<string>` (in `Accordion`)	Which items are open. One shape for both modes: single mode just never holds more than one id
`headerRefs: useRef([])` (in `Accordion`)	DOM nodes of the header buttons, to move focus on arrow keys (not state: changing it must not re-render)
`allowMultiple` (in `App`)	Demo toggle; a **prop** for `Accordion`

```
const [openIds, setOpenIds] = useState(
  () => new Set(allowMultiple ? defaultOpenIds : defaultOpenIds.slice(0, 1))
);
const headerRefs = useRef([]);
```

Senior note: derive, don't store

- `isOpen` for an item is **derived**: `openIds.has(item.id)`. No `isOpen` flag inside each item, so the two modes can't disagree.
- Panel **height is not state** at all. CSS works it out (Challenge 1). Storing a measured height is how the original scaffold went wrong.
- ARIA ids are **derived** from `useId()`, not stored.
- Why not `activeIndex: number`? It only supports single mode, and an index breaks when items are reordered, added or removed. Ids are stable.
- **Ownership:** `Accordion` owns `openIds` (only it changes them). `AccordionItem` owns nothing; it gets `isOpen` and callbacks. `App` owns the mode.

5. Design the data structures and contracts

Example data

```
export const faqItems = [
  { id: "what-is-react", title: "What is React?", content: "React is a JavaScript library..." },
  { id: "what-is-component", title: "What is a component?", content: "A component is..." },
```

```
  // ...
];
```

- **Stable string `id`**: used as the React `key` and stored in `openIds`. Using the array index would open the wrong item after a reorder or a delete.
- `title` is a string (it becomes the accessible name of the button). `content` can be any React node.

Component contract (no API, so the props are the contract)

```
<Accordion
  items={[{ id, title, content }]}   // required; [] shows the empty state
  allowMultiple={false}               // optional, default false
  defaultOpenIds={["what-is-react"]}  // optional, read once on mount
/>
```

```
<AccordionItem
  item={{ id, title, content }}
  isOpen={true}
  onToggle={() => {}}
  onKeyDown={(e) => {}}
  headerRef={(el) => {}}
/>
```

Pure helpers:

Function	Input	Output
`toggleId(openIds, id, allowMultiple)`	`Set{"a"}, "b", false`	`Set{"b"}` (new Set)
	`Set{"a"}, "b", true`	`Set{"a","b"}`
	`Set{"a"}, "a", false`	`Set{}`
`nextIndex(key, index, count)`	`"ArrowDown", 3, 4`	`0` (wraps)
	`"Home", 2, 4`	`0`
	`"Enter", 0, 4`	`null` (not ours)

6. Trace the data flow

Single mode, "What is React?" is open, user clicks "What is useState?":

```
click on <button> "What is useState?"
        |
        ▼
AccordionItem → onToggle()  →  Accordion.handleToggle("what-is-usestate")
        |
        ▼
```

```
setOpenIds(prev => toggleId(prev, "what-is-usestate", false))
        │   prev = {"what-is-react"}  →  start from empty Set (single mode)
        │   "what-is-usestate" was not open → add it
        ▼
openIds = {"what-is-usestate"}   (new Set → React re-renders Accordion)
        │
        ▼
items.map: isOpen = openIds.has(item.id)
        │   React item: isOpen false → aria-expanded="false", panel loses .is-open, inert
        │   useState item: isOpen true → aria-expanded="true", panel gets .is-open
        ▼
CSS transitions grid-template-rows: 1fr → 0fr (React panel) and 0fr → 1fr (useState panel)
chevron rotates via [aria-expanded="true"]
        │
        ▼
Screen reader: "What is useState?, button, expanded"
```

7. Important technical challenges

Challenge 1: Animating to `height: auto`

Problem: CSS can't transition `height: 0` → `height: auto`. The common fixes break:

- `max-height: 1000px` hack: the timing is wrong (it animates to 1000px, so closing a 100px panel waits ~90% of the time doing nothing), and content taller than 1000px is cut off.
- Measuring `scrollHeight` in JS: the original scaffold called `useRef` **inside** `.map()` (breaks the Rules of Hooks: hooks must run in the same order every render) and read `contentRef.current?.scrollHeight` during render, which is `undefined` on the first render and goes stale if the content grows (an image loads, the window narrows and text wraps) → content gets clipped.

Solution: Animate a **grid row** from `0fr` to `1fr`. The browser knows the real content height, so no measuring, and it stays correct when the content changes. The inner wrapper needs `min-height: 0` + `overflow: hidden` so the row can shrink to 0.

```
.accordion-panel { display: grid; grid-template-rows: 0fr; transition: grid-template-rows 250ms ease; }
.accordion-panel.is-open { grid-template-rows: 1fr; }
.accordion-panel-inner { overflow: hidden; min-height: 0; }
.accordion-content { padding: 0 16px 16px; } /* padding inside, so 0fr really is 0px */
```

Challenge 2: One state shape for single and multiple mode

Problem: `activeIndex` handles only single mode. Two separate states (`activeIndex` + `openSet`) can disagree. And if the parent switches from multiple to single while 3 panels are open, single mode now shows 3 open panels.

Solution: Always a `Set` of ids, with one pure rule. In single mode we start from an empty Set, so at most one id survives. Return a **new** Set so React sees a new reference and re-renders. When the mode changes, the parent remounts the accordion with a `key` .

```
export function toggleId(openIds, id, allowMultiple) {
  const next = new Set(allowMultiple ? openIds : []);
  if (openIds.has(id)) next.delete(id);
  else next.add(id);
  return next;
}
```

```
<Accordion key={String(allowMultiple)} allowMultiple={allowMultiple} ... />
```

Challenge 3: Accessibility (semantics + hidden content)

Problem: A `<div onClick>` header can't be focused with Tab, doesn't react to Enter / Space, and is announced as plain text. A collapsed panel animated to 0px is still in the DOM, so Tab lands on invisible links inside it and screen readers read the hidden text. Hard-coded ids (`panel-1`) collide when two accordions are on one page.

Solution: A real `<button type="button">` inside a heading (`<h3>`) gives focus, Enter / Space and the heading outline for free. `aria-expanded` tells the state, `aria-controls` + `aria-labelledby` link header and panel, `useId()` makes ids unique per instance, and `inert` removes collapsed content from Tab order and the accessibility tree while keeping it in the DOM (so it can still animate).

```
<h3 className="accordion-heading">
  <button id={headerId} type="button" aria-expanded={isOpen} aria-controls={panelId} onClick={onToggle}>
    {item.title}
  </button>
</h3>
<div id={panelId} role="region" aria-labelledby={headerId} inert={!isOpen}>...</div>
```

Style the chevron from `[aria-expanded="true"]` , so the visual and the announced state can never drift apart.

Challenge 4: Keyboard navigation between headers

Problem: The WAI-ARIA accordion pattern expects Arrow Up / Down to move between headers (wrapping) and Home / End to jump to the first / last. We need to move **focus**, which is a DOM operation, and not scroll the page.

Solution: Keep the header nodes in one `useRef([])` array (filled by a ref callback per item), compute the target with a pure `nextIndex` , and `preventDefault()` only for keys we handle.

```
const handleKeyDown = (e, index) => {
  const next = nextIndex(e.key, index, items.length);
  if (next === null) return;
  e.preventDefault(); // stop Home/End/arrows from scrolling the page
  headerRefs.current[next]?.focus();
};
```

8. Implement in the correct order

#	Step	Working result
1	Data file with stable ids; render titles + content with `.map`	Static list
2	`openIds` Set state; show content only when `openIds.has(id)`	Click toggles (multiple mode behavior)
3	Extract `toggleId` with the `allowMultiple` rule	Single mode works
4	Node `assert` check for `toggleId`	Rule proven
5	Split `AccordionItem`; header becomes `<h3><button>`	Tab + Enter / Space work
6	`useId` ids + `aria-expanded`, `aria-controls`, `role="region"`, `aria-labelledby`	Screen readers announce state
7	Always render the panel; grid `0fr → 1fr` animation; chevron from `aria-expanded`	Smooth height animation
8	`inert={!isOpen}` on the panel	Tab skips collapsed content
9	`headerRefs` + `nextIndex` + `onKeyDown`	Arrow / Home / End navigation
10	Empty state, `prefers-reduced-motion`, focus ring	Edge cases
11	Demo: "Allow multiple open" checkbox with `key` reset	Both modes demoed
12	Walk through the use-case table manually	Done

9. Cover all states and edge cases

State / edge case	What happens
Initial	Items from `defaultOpenIds` open (first one only in single mode), rest closed
Empty (`items = []`)	"Nothing to show yet." message instead of an empty bordered box
All closed	Allowed; every header shows the down chevron
One open (single mode)	Opening another closes it in the same render: no frame where two are open
Many open (multiple mode)	Each toggles independently
Mode switch (multiple → single with 3 open)	`key` remounts → back to the default state
`defaultOpenIds` has an id that doesn't exist	Ignored: no item matches, nothing breaks
Long content / content grows after opening	Grid `1fr` follows the real height; nothing clipped
Rapid clicks during the animation	CSS transition reverses from the current height
Collapsed panel with links / inputs	`inert`: not focusable, not read out

State / edge case	What happens
Two accordions on one page	`useId` keeps every id unique
Arrow keys on first / last header	Wrap around
Home / End / arrows	`preventDefault`, so the page doesn't scroll
Reduced motion	Transitions turned off
Very long title	Wraps inside the button; chevron has `flex-shrink: 0`

10. Final structure

```
_17_accordion/
├─ index.html
├─ package.json
└─ src/
   ├─ main.jsx
   ├─ index.css             # reset + body
   ├─ App.jsx               # demo page + mode toggle
   ├─ App.css               # accordion styles + animation
   ├─ data/
   │  └─ faq.js             # example items with stable ids
   ├─ utils/
   │  └─ accordion.js       # toggleId, nextIndex (pure)
   └─ components/
      ├─ Accordion.jsx      # state, keyboard nav, list
      └─ AccordionItem.jsx  # button + panel + ARIA
check.mjs                   # node assert self-check of the pure rules
```

11. Full working code

`main.jsx` stays as Vite generated it (renders `<App />` inside `StrictMode` and imports `index.css`).

`src/utils/accordion.js`

```
// Returns a NEW Set (never mutate state). Single mode: opening one closes the rest.
export function toggleId(openIds, id, allowMultiple) {
  const next = new Set(allowMultiple ? openIds : []);
  if (openIds.has(id)) next.delete(id);
  else next.add(id);
  return next;
}

// Which header gets focus for a key press; null = not a navigation key.
export function nextIndex(key, index, count) {
  switch (key) {
```

```
    case "ArrowDown":
      return (index + 1) % count;
    case "ArrowUp":
      return (index - 1 + count) % count;
    case "Home":
      return 0;
    case "End":
      return count - 1;
    default:
      return null;
  }
}
```

src/data/faq.js

```
export const faqItems = [
  {
    id: "what-is-react",
    title: "What is React?",
    content: "React is a JavaScript library for building user interfaces out of components.",
  },
  {
    id: "what-is-component",
    title: "What is a component?",
    content: "A component is a reusable piece of UI: a function that takes props and returns JSX.",
  },
  {
    id: "what-is-usestate",
    title: "What is useState?",
    content:
      "useState is a hook that gives a function component its own state. Calling the setter schedules a
re-render with the new value. Updates are batched, so when the next value depends on the previous one,
pass an updater function: setCount((c) => c + 1).",
  },
  {
    id: "what-is-key",
    title: "Why do lists need a key?",
    content: "Keys tell React which item is which between renders, so it keeps the right state on the
right item.",
  },
];
```

src/components/AccordionItem.jsx

```
import { useId } from "react";
```

```
export default function AccordionItem({ item, isOpen, onToggle, onKeyDown, headerRef }) {
  const id = useId();
  const headerId = `${id}-header`;
  const panelId = `${id}-panel`;

  return (
    <div className="accordion-item">
      <h3 className="accordion-heading">
        <button
          ref={headerRef}
          id={headerId}
          type="button"
          className="accordion-trigger"
          aria-expanded={isOpen}
          aria-controls={panelId}
          onClick={onToggle}
          onKeyDown={onKeyDown}
        >
          <span>{item.title}</span>
          <span className="accordion-icon" aria-hidden="true" />
        </button>
      </h3>

      {/* inert: collapsed content can't be tabbed into or read by screen readers */}
      <div
        id={panelId}
        role="region"
        aria-labelledby={headerId}
        className={`accordion-panel${isOpen ? " is-open" : ""}`}
        inert={!isOpen}
      >
        <div className="accordion-panel-inner">
          <div className="accordion-content">{item.content}</div>
        </div>
      </div>
    </div>
  );
}
```

src/components/Accordion.jsx

```
import { useRef, useState } from "react";
import AccordionItem from "./AccordionItem";
import { nextIndex, toggleId } from "../utils/accordion";

export default function Accordion({ items, allowMultiple = false, defaultOpenIds = [] }) {
  const [openIds, setOpenIds] = useState(
```

```
    () => new Set(allowMultiple ? defaultOpenIds : defaultOpenIds.slice(0, 1))
  );
  const headerRefs = useRef([]);

  if (items.length === 0) {
    return <p className="accordion-empty">Nothing to show yet.</p>;
  }

  const handleToggle = (id) => setOpenIds((prev) => toggleId(prev, id, allowMultiple));

  const handleKeyDown = (e, index) => {
    const next = nextIndex(e.key, index, items.length);
    if (next === null) return;
    e.preventDefault(); // stop Home/End/arrows from scrolling the page
    headerRefs.current[next]?.focus();
  };

  return (
    <div className="accordion">
      {items.map((item, index) => (
        <AccordionItem
          key={item.id}
          item={item}
          isOpen={openIds.has(item.id)}
          onToggle={() => handleToggle(item.id)}
          onKeyDown={(e) => handleKeyDown(e, index)}
          headerRef={(el) => {
            headerRefs.current[index] = el;
          }}
        />
      ))}
    </div>
  );
}
```

src/App.jsx

```
import { useState } from "react";
import Accordion from "./components/Accordion";
import { faqItems } from "./data/faq";
import "./App.css";

export default function App() {
  const [allowMultiple, setAllowMultiple] = useState(false);

  return (
    <main className="page">
```

```
      <h1>Accordion</h1>

      <label className="mode-toggle">
        <input
          type="checkbox"
          checked={allowMultiple}
          onChange={(e) => setAllowMultiple(e.target.checked)}
        />
        Allow multiple open
      </label>

      {/* key: switching mode remounts the accordion so its open state resets */}
      <Accordion
        key={String(allowMultiple)}
        items={faqItems}
        allowMultiple={allowMultiple}
        defaultOpenIds={["what-is-react"]}
      />
    </main>
  );
}
```

src/index.css

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: system-ui, "Segoe UI", Arial, sans-serif;
  color: #1f2937;
  background: #f8fafc;
  -webkit-font-smoothing: antialiased;
}
```

src/App.css

```
.page { max-width: 560px; margin: 48px auto; padding: 0 16px; }
.page h1 { text-align: center; font-size: 32px; margin-bottom: 16px; }
.mode-toggle { display: flex; align-items: center; gap: 8px; margin-bottom: 16px; cursor: pointer; }

.accordion { border: 1px solid #e2e8f0; border-radius: 8px; background: #fff; overflow: hidden; }
.accordion-item + .accordion-item { border-top: 1px solid #e2e8f0; }
.accordion-heading { font-size: 16px; }
```

```
.accordion-trigger {
  width: 100%;
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 12px;
  padding: 14px 16px;
  border: 0;
  background: none;
  color: inherit;
  font: inherit;
  font-weight: 600;
  text-align: left;
  cursor: pointer;
}
.accordion-trigger:hover { background: #f1f5f9; }
.accordion-trigger:focus-visible { outline: 2px solid #2563eb; outline-offset: -2px; }

/* chevron styled from aria-expanded, so visuals can't drift from the a11y state */
.accordion-icon {
  flex-shrink: 0;
  width: 8px;
  height: 8px;
  border-right: 2px solid currentColor;
  border-bottom: 2px solid currentColor;
  transform: rotate(45deg);
  transition: transform 250ms ease;
}
.accordion-trigger[aria-expanded="true"] .accordion-icon { transform: rotate(-135deg); }

/* 0fr -> 1fr animates to the content's real height, no measuring in JS */
.accordion-panel { display: grid; grid-template-rows: 0fr; transition: grid-template-rows 250ms ease; }
.accordion-panel.is-open { grid-template-rows: 1fr; }
.accordion-panel-inner { overflow: hidden; min-height: 0; }
.accordion-content { padding: 0 16px 16px; color: #475569; line-height: 1.6; }

.accordion-empty { padding: 16px; text-align: center; color: #64748b; }

@media (prefers-reduced-motion: reduce) {
  .accordion-panel,
  .accordion-icon { transition: none; }
}
```

`check.mjs` **(run with** `node check.mjs` **)**

```
import assert from "node:assert/strict";
import { nextIndex, toggleId } from "./src/utils/accordion.js";

const ids = (s) => [...s].sort();

// single mode: opening one closes the other; clicking the open one closes it
const s = new Set(["a"]);
assert.deepEqual(ids(toggleId(s, "b", false)), ["b"]);
assert.deepEqual(ids(toggleId(s, "a", false)), []);
// multiple mode: independent toggles
assert.deepEqual(ids(toggleId(s, "b", true)), ["a", "b"]);
assert.deepEqual(ids(toggleId(new Set(["a", "b"]), "a", true)), ["b"]);
// never mutates the input
assert.deepEqual(ids(s), ["a"]);

// keyboard: wraps at both ends, Home/End, ignores other keys
assert.equal(nextIndex("ArrowDown", 3, 4), 0);
assert.equal(nextIndex("ArrowUp", 0, 4), 3);
assert.equal(nextIndex("ArrowDown", 1, 4), 2);
assert.equal(nextIndex("Home", 2, 4), 0);
assert.equal(nextIndex("End", 0, 4), 3);
assert.equal(nextIndex("Enter", 0, 4), null);
console.log("accordion checks passed");
```

12. Wrap-up

Edge cases handled

- Single and multiple mode with one state shape; closing all; mode switch resets cleanly.
- Content of any height, including content that changes after opening.
- Native button semantics, full ARIA wiring, unique ids per instance, collapsed content made `inert`.
- Arrow / Home / End navigation with wrap-around and no page scroll.
- Empty list, reduced motion, long titles.

Trade-offs to mention

- **Grid** `0fr → 1fr` vs measuring `scrollHeight`: no JS and always correct, but needs an extra wrapper div and a modern browser (Chrome 107+, Safari 16+, Firefox 66+). Older browsers just open without animation, which is fine.
- **Always rendering panels** (needed for the closing animation) vs mounting only open ones: content of closed panels is in the DOM. Fine for FAQs; for heavy content, lazy-mount on first open.
- **Uncontrolled state** keeps the API small; a parent that needs to save the open state (URL, local storage) needs controlled mode.
- `role="region"` **on every panel** is fine for a few items; with many (6+) it floods the landmark list, so drop it there.

- **Native** `<details>/<summary>` would give toggling and find-in-page for free, but single mode and the height animation are harder to control.

What to add with more time

- Controlled mode: `openIds` + `onChange` props.
- Disabled items (`aria-disabled` , skipped by arrow keys).
- Lazy-mount heavy panels on first open.
- Deep links: open the item whose id is in `location.hash` and scroll to it.
- Tests: React Testing Library for click / keyboard / ARIA, plus axe for accessibility.

13. Counter questions the interviewer may ask

Requirements

Q1. Why not just use `<details>` and `<summary>`?

They give a toggle, keyboard support and browser find-in-page for free, and are a good answer for a simple FAQ. Modern browsers even support single-open mode through a shared `name` attribute. But animating `<details>` closing is awkward because the content is hidden the moment it closes, and you can't add Arrow / Home / End navigation or a controlled mode without JS anyway. A custom component gives full control over both.

Q2. Should one panel always stay open in single mode?

Only if the product asks for it. The APG allows both. To force it, make `toggleId` return `openIds` unchanged when the clicked id is the only open one, and set `aria-disabled="true"` on that header so screen readers know it can't be collapsed.

State design

Q3. Why a `Set` instead of an array of open ids or `activeIndex`?

`has()` is O(1) and reads clearly, duplicates are impossible, and one shape covers both modes. `activeIndex` only supports one open item and breaks when items are reordered. An array works too, but you'd need `includes` and to guard against duplicates.

Q4. React compares state by reference. Is mutating the Set a problem?

Yes. `prev.add(id); return prev;` returns the same object, so React skips the re-render. `toggleId` always builds a `new Set(...)` , and the self-check asserts the input isn't mutated.

Q5. Why does `Accordion` own the state and not each `AccordionItem`?

In single mode, opening one item must close another, so the state has to live in their common parent. Item-level `isOpen` would need items to talk to each other. Items stay simple: they get `isOpen` and callbacks.

Q6. How would you make it controlled?

Accept `openIds` and `onChange` props. If `openIds` is passed, use it and call `onChange(toggleId(openIds, id, allowMultiple))` ; otherwise use internal state. Same pattern as `<input value onChange>` vs `defaultValue` .

```
const isControlled = openIdsProp !== undefined;
const openIds = isControlled ? openIdsProp : internalOpenIds;
```

Q7. Why `key={String(allowMultiple)}` in `App`?

`defaultOpenIds` and the mode are read once into state. Changing the `key` remounts `Accordion`, so state is rebuilt from the new props. Without it, switching from multiple to single could leave 3 panels open in single mode.

Animation

Q8. Why not the `max-height` trick the scaffold used?

You must guess a max value: too small clips content, too large makes the timing wrong (closing looks delayed because most of the transition is spent above the real height). Grid `0fr → 1fr` animates to the real height with no guessing.

Q9. Why not measure `scrollHeight` with a ref?

It works but needs a ref per item, has to re-measure when content changes (`ResizeObserver`), and reading layout in render is wrong (refs are `null` on the first render). The scaffold also called `useRef` inside `.map()`, which breaks the Rules of Hooks: if the number of items changes, hooks shift positions and React throws or mixes up state.

Q10. framer-motion is installed. Why not `motion.div` with `animate={{ height: "auto" }}`?

It works well and handles `auto` height for you, and is a fair choice if the project already uses it. But it pulls a sizeable library into the bundle for one effect CSS can do, and it runs JS on every frame. Mention it as an option, pick CSS by default.

Q11. Is animating `grid-template-rows` bad for performance?

It triggers layout every frame, like animating `height`, but only for that one panel and only for 250 ms. That's fine for an accordion. Pure `transform` animations would be cheaper but can't change the space the panel takes in the page.

Accessibility

Q12. Why `inert` instead of `hidden` or `display: none`?

`display: none` / `hidden` removes the content instantly, so there's nothing to animate when closing. `inert` keeps it rendered (so the height can animate) but makes it unfocusable and hidden from screen readers. React 19 supports `inert` as a boolean prop.

Q13. Why wrap the button in an `<h3>`?

Screen reader users jump between headings to scan a page. The heading level should fit the page outline (make it a prop if the accordion is used under different heading levels). The button goes **inside** the heading, never the other way round.

Q14. Why `useId()` instead of `` `panel-${item.id}` ``?

`item.id` is only unique within one list. Two accordions on the same page showing the same item would create duplicate DOM ids and break `aria-controls`. `useId` is unique per component instance and stable between server and client rendering.

Q15. Do I need to handle Enter and Space myself?

No. A native `<button>` fires `click` for both. That's the main reason not to use a `<div role="button">`, which would need `tabIndex`, a key handler, and correct Space behavior (fires on key up).

Keyboard

Q16. Why refs for focus, and not state like `focusedIndex`?

Focus lives in the DOM; mirroring it in state causes extra renders and can drift from the real `document.activeElement` (for example when the user clicks). A ref array plus `.focus()` moves real focus directly.

Q17. Why is the ref callback written with a block body?

In React 19 a ref callback may return a **cleanup function**. `(el) => (refs.current[i] = el)` returns `el`, which reads like a cleanup and TypeScript rejects it. The block body returns nothing, so the intent is clear.

Scale and "what if"

Q18. What if there are 1,000 items?

Toggling is O(1) (Set lookup), but rendering 1,000 items with their content is heavy. Lazy-mount panel content on first open, wrap `AccordionItem` in `memo` with stable callbacks (pass `id` and a single `onToggle(id)` instead of a new arrow per item), and if the list itself is huge, virtualize it, keeping focus management in mind.

Q19. What if the content comes from an API when the panel opens?

Fetch on first open, keep `{ status, data, error }` per id (a `Map` or object keyed by id), show a spinner inside the panel while loading and a retry button on error. Cache the result so reopening doesn't refetch. The grid animation handles the content size changing from spinner to data.

Q20. What if a user lands on `/faq#what-is-usestate`?

On mount, read `location.hash`, add that id to `openIds` (via `defaultOpenIds` or controlled mode), and `scrollIntoView()` the header after the render. Update the hash when a panel opens if shareable links are wanted.

Q21. Does this work with nested accordions?

Yes. Each nested `Accordion` has its own state and its own `useId` ids. CSS uses `.accordion-panel.is-open` on the panel itself and `[aria-expanded]` on the button itself, not a parent `.is-open` class, so an open outer item doesn't style the inner ones as open.

Testing and trade-offs

Q22. How would you test it?

Unit-test `toggleId` / `nextIndex` (pure, done in `check.mjs`). With React Testing Library: click a header → `aria-expanded="true"` and, in single mode, the other header becomes `"false"`; `userEvent.keyboard("{ArrowDown}")` moves focus; `getByRole("button", { name: "What is React?" })` proves the accessible name. Run `jest-axe` for accessibility rules.

Q23. Why not one big component instead of `Accordion` + `AccordionItem`?

Each item needs its own `useId`, and hooks can't be called inside `.map()`. A separate component is the clean way to give each item hooks. It also keeps the parent focused on state and keyboard logic.